

Universidade Estadual do Piauí - UESPI

Centro de Tecnologia e Urbanismo

Bacharelado em Ciência da Computação

Disciplina: Sistemas Distribuídos - 3.^a Avaliação

Relatório Técnico: Plataforma de Pedidos Online

Autor: Thiago Costa Sousa

Data: Junho de 2026

1. Introdução e Visão Geral

Neste relatório, descrevo o desenvolvimento de um sistema distribuído que simula uma plataforma de e-commerce e delivery. O objetivo principal foi colocar em prática os conceitos vistos na disciplina de Sistemas Distribuídos, como comunicação entre processos, sincronização, tolerância a falhas e, claro, a arquitetura de microsserviços.

Para tirar o projeto do papel, utilizei Node.js e Docker Compose. Essa escolha foi fundamental para rodar cada serviço como um processo independente, isolado em seus próprios containers e portas, garantindo que o sistema fosse distribuído tanto na teoria quanto na prática.

2. Arquitetura de Microsserviços e Separação de Responsabilidades

A arquitetura foi desenhada em torno de quatro microsserviços principais, além de uma infraestrutura robusta de mensageria e persistência. Um ponto importante do projeto foi evitar o compartilhamento de bancos de dados entre os serviços, garantindo um desacoplamento real, como deve ser em uma arquitetura de microsserviços.

Serviço	Responsabilidade	Tecnologias
Gateway / API Principal	É a porta de entrada. Ele organiza as requisições HTTP, cuida dos <i>timeouts</i> e	Node.js, Express, Axios

Serviço	Responsabilidade	Tecnologias
	gerencia as transações compensatórias quando algo dá errado.	
Serviço de Estoque	Fica responsável por controlar o estoque dos produtos, cuidando de reservas e fazendo <i>rollbacks</i> quando necessário. Cada instância tem seu próprio banco.	Node.js, Express, PostgreSQL
Serviço de Pedidos	Processa a compra em si e avisa o sistema que o pedido foi concluído via mensagens. Também opera com banco de dados isolado.	Node.js, Express, PostgreSQL
Serviço de Notificações	Funciona de forma assíncrona, consumindo as filas de mensagens para disparar alertas (logs/e-mails simulados) sem travar a requisição do usuário.	Node.js, amqplib

3. Modelos de Comunicação Distribuída

Para atender aos requisitos de invocação remota e comunicação indireta, a plataforma implementa um modelo híbrido:

- **Comunicação Síncrona (REST API):** Utilizada nas interações diretas onde a resposta imediata é essencial para o fluxo. O Gateway atua como um cliente REST, invocando os endpoints do Serviço de Estoque e do Serviço de Pedidos via protocolo HTTP/TCP.
- **Comunicação Assíncrona e Indireta (Publish-Subscribe):** Uma vez que o Serviço de Pedidos grava a transação no banco com sucesso, ele atua como *publisher* enviando o evento `pedido_criado` para um *broker RabbitMQ*. O Serviço de Notificações atua como *subscriber* (consumidor). Essa abordagem reduz o acoplamento temporal: o Serviço de Pedidos não precisa saber se o de Notificações está ativo no momento exato da compra.

4. Sincronização e Concorrência Distribuída

Um dos problemas em plataformas de e-commerce é a condição de corrida (*race condition*) ao atualizar estoques, o que pode resultar em valores negativos caso dois clientes tentem comprar a última unidade simultaneamente.

Para solucionar este problema de forma robusta e distribuída, evitei o uso de primitivas de sincronização em memória (como Mutex de processo único), adotando um mecanismo de bloqueio no nível da persistência de dados. A implementação utiliza a estratégia de **Transação SQL com Bloqueio de Linha (Pessimistic Locking)** no banco PostgreSQL do Serviço de Estoque.

```
BEGIN;  
SELECT quantidade FROM produtos WHERE id = $1 FOR UPDATE;  
-- A cláusula FOR UPDATE atua como um lock distribuído.  
-- Outras requisições aguardam até o COMMIT ou ROLLBACK.  
UPDATE produtos SET quantidade = quantidade - $2 WHERE id = $1;  
COMMIT;
```

Essa abordagem garante a exclusão mútua distribuída, assegurando a integridade dos dados mesmo com escalabilidade horizontal dos nós de serviço.

5. Tolerância a Falhas e Consistência

A ocorrência de falhas parciais é inerente a qualquer sistema distribuído. O projeto lida com três cenários principais de anomalias:

- **Timeouts para Atrasos de Rede:** O Gateway implementa timeouts curtos (ex: 3000ms) em suas requisições externas para evitar bloqueios indefinidos (*thread starvation*) caso os serviços de retaguarda sofram lentidão.
- **Consistência Eventual via Padrão Saga (Compensação):** Se o Serviço de Estoque realiza a reserva com sucesso, mas a comunicação com o Serviço de Pedidos falha em

seguida, o sistema ficaria em um estado inconsistente. O Gateway intercepta essa falha e dispara um *Rollback* (operação compensatória) chamando o *endpoint* de cancelamento de reserva no Serviço de Estoque.

- **Idempotência no Consumo de Mensagens:** Operando sob o princípio de entrega *at-least-once* do RabbitMQ, podem ocorrer reentregas de mensagens em casos de instabilidade da rede. O Serviço de Notificações valida o ID do pedido processado para assegurar que alertas duplicados sejam ignorados de maneira segura.

6. Metodologia de Simulação para Testes

O sistema conta com mecanismos preparados para facilitar a demonstração prática das vulnerabilidades abordadas:

- **Simulação de Atraso (Delay):** Rotas específicas nos microsserviços permitem a injeção artificial de pausas (*sleep*), viabilizando o teste empírico do comportamento do Gateway frente aos *timeouts*.
- **Simulação de Queda de Serviços:** A orquestração via Docker Compose possibilita a interrupção abrupta de um *container* específico (ex: `docker-compose stop servico_pedidos`). Isso comprova a eficácia das lógicas de *rollback* do Gateway ou a persistência das mensagens no RabbitMQ, que mantém os eventos enfileirados até a restauração do Serviço de Notificações.

7. Conclusão

O projeto consolida os fundamentos acadêmicos em uma aplicação prática moderna. Ao aliar a separação rígida de *containers* e instâncias de banco de dados a estratégias avançadas de compensação de transações e **locking** pessimista no SGBD, o ambiente resultante reflete os desafios enfrentados pela engenharia de software distribuída corporativa contemporânea.